Universidade Cruzeiro do Sul

TÓPICOS AVANÇADOS EM SISTEMAS DE INFORMAÇÃO II

Prof. José Claudio



Graduado em Ciência da Computação(2011)

- Especialização em Docência do Ensino Superior(2020)

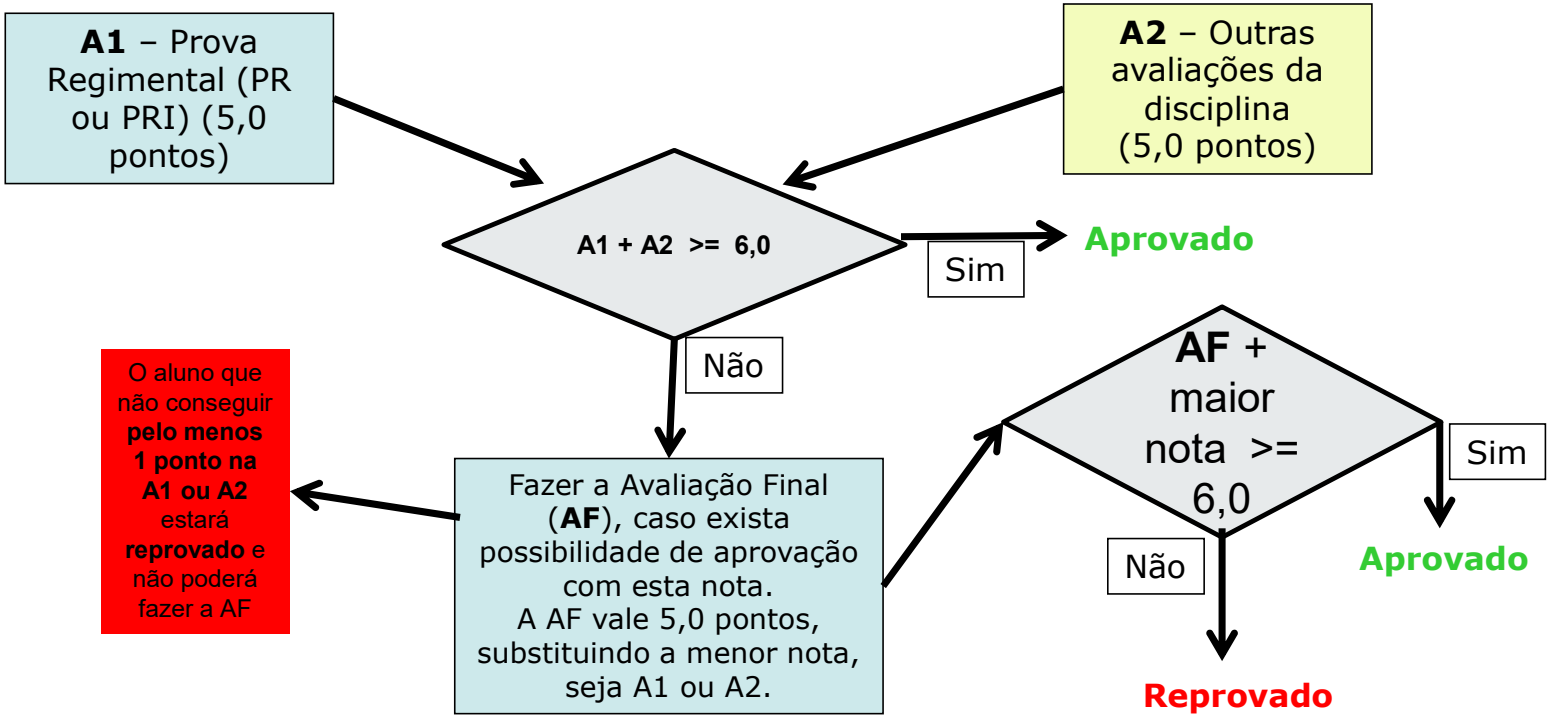
Cursos em que ministro

- Análise e Desenvolvimento de Sistemas, Ciência da Computação

TÓPICOS AVANÇADOS EM SISTEMA DE INFORMAÇÃO II

OBJETIVOS	
Cognitivos	Estudo dos principais conceitos e processos avançados envolvidos na Engenharia de Software, Gestão de Serviços de TI e na Gestão e Auditoria de TI
Habilidades	Planejar a implantação da Gestão de Serviços de TI e na Gestão e auditoria de TI
Atitudes	Dominar os principais conceitos e processos avançados de Engenharia de Software, Gestão de Serviços de TI e na Gestão e auditoria de TI

UNID.	C/H	Conteúdo
I	3	**Apresentação da metodologia** Apresentação da metodologia; Plano de ensino; Processo de avaliação e revisão da bibliografia
II	12	**Engenharia de Software** Tópicos avançados de DEVOPS; SCRUM e Design Thinks na Prática.
III	9	**Arquitetura de Desenvolvimento de Software** Padrão de Arquitetura de Software, Design Pattenrs, Boas Prática de Arquitetura de Software.
IV	12	**Segurança da Informação** Segurança da Informação e Gestão de Riscos; LGPD; Desenvolvimento de Software Seguro.
V	12	**Estratégia e Gestão de Serviços de TI** Modelo ITIL; Engenharia e Qualidade de Software.
VI	12	**Elaboração de artigos científico para publicação** Aplicação dos conceitos de governança em artigos científicos para publicação.



Reprovação se a frequência for inferior a 75% (setenta e cinco por cento)

Slide, Prof. Claudio,2024

Universidade Cruzeiro do Sul

Tópicos Avançados de DevOps

Universidade Cruzeiro do Sul

Tópicos Avançados de DevOps

DevOps é uma abordagem que combina práticas, ferramentas e filosofias culturais para aumentar a capacidade de uma organização de entregar aplicativos e serviços em alta velocidade. DevOps visa melhorar a colaboração entre equipes de desenvolvimento (Dev) e operações (Ops), promovendo uma cultura de automação, integração contínua e entrega contínua.

Importância de DevOps

DevOps surgiu para resolver a tradicional divisão entre as equipes de desenvolvimento de software e operações de TI, onde a falta de comunicação e alinhamento causava atrasos e ineficiências no ciclo de vida do software. Com DevOps, as organizações podem acelerar a entrega de software, melhorar a qualidade do código e aumentar a satisfação do cliente.

Cultura de DevOps

Colaboração e Comunicação

- A cultura DevOps promove um ambiente onde as equipes de desenvolvimento, operações, e outros departamentos trabalham juntos em um ciclo de feedback contínuo.
- Incentiva a transparência, a responsabilidade compartilhada e a propriedade conjunta do produto ao longo de todo o ciclo de vida de desenvolvimento.

Cultura de DevOps

Mentalidade de Aprendizado Contínuo

- Em DevOps, erros são vistos como oportunidades de aprendizado, e há uma ênfase em experimentação e melhoria contínua.
- Estabelecer uma mentalidade de "construir, medir, aprender" é fundamental para o sucesso.

Cultura de DevOps

Automação e Ferramentas

A automação é um pilar da cultura DevOps. Desde a integração até o deploy e monitoramento, a automação ajuda a reduzir erros humanos e acelera processos repetitivos.

Ferramentas populares incluem Jenkins, Docker, Kubernetes, Ansible, Terraform, entre outras.

Cultura de DevOps

Automação e Ferramentas

Jenkins: ferramenta de automação de CI/CD

Docker: plataforma de containers para empacotar aplicações e dependências.

Kubernetes: Orquestrador de containers para gerenciar aplicações em escala.

Cultura de DevOps

Automação e Ferramentas

Ansible: Ferramenta de automação de configuração via SSH

Terraform: ferramenta de IAC para provisionar recursos em nuvem/local.

Principais práticas de DevOps

Integração Contínua (CI - Continuous Integration)

Uma prática onde os desenvolvedores frequentemente juntam seu código em um repositório central, onde são executados testes automáticos.

A integração contínua tem por objetivo detectar erros rapidamente, melhorar a qualidade do software e reduzir o tempo necessário para validar e lançar novas atualizações.

Principais práticas de DevOps

Entrega Contínua (CD - Continuous Delivery)

A prática de garantir que o código está em um estado que pode ser lançado a qualquer momento, isso faz com que o processo de lançamento de software se torne mais rápido, seguro e eficiente, facilitando lançamentos frequentes.

Principais práticas de DevOps

Monitoramento e Logging

É a prática de monitorar continuamente as aplicações em produção para detectar problemas em tempo real.

- **Ferramentas:** Prometheus, Grafana, ELK Stack (Elasticsearch, Logstash, Kibana).

Principais práticas de DevOps

Ferramentas de Monitoramento e Logging

Prometheus, Grafana, ELK Stack (Elasticsearch, Logstash, Kibana).

Principais práticas de DevOps

Infraestrutura como Código (IaC - Infrastructure as Code)

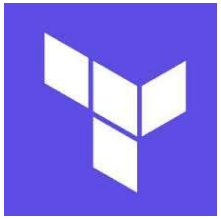
Infraestrutura como Código é a prática de gerenciar e provisionar data centers computacionais através de arquivos de configuração legíveis por máquina, em vez de configuração física ou ferramentas de gerenciamento interativas.

Ferramentas: Terraform, Ansible, AWS CloudFormation.

Universidade Cruzeiro do Sul

Principais práticas de DevOps

Ferramentas para Infraestrutura como Código

Terraform: 

Ansible: 

Principais práticas de DevOps

Gestão de Configuração:

- **O que é:** Gerenciar as configurações de sistemas e software de forma automatizada.
- **Ferramentas:** Puppet, Chef, SaltStack.

Pipeline de DevOps

Um pipeline de DevOps é a automação completa do processo de desenvolvimento, isso inclui buil, teste, e deploy do software. Ele inclui CI/CD, testes automatizados, revisão de código, deploy automatizado e monitoramento.

Pipeline de DevOps

Componentes Principais:

1. **Versionamento de Código (Git):** Armazena e controla as versões do código-fonte.
2. **Build Automation (Jenkins, Travis CI):** Compila e empacota o código em artefatos distribuíveis.
3. **Testes Automatizados:** Validam o código para garantir sua qualidade e funcionalidade.
4. **Deploy Automatizado (Kubernetes, AWS):** Implementa o software nos ambientes de produção.
5. **Monitoramento e Feedback:** Garante que os problemas sejam rapidamente identificados e corrigidos.

Ferramentas de DevOps

Jenkins:

Automação de integração e entrega contínua.

Docker:

Contêinerização de aplicativos para garantir que funcionem de forma consistente em diferentes ambientes.

Ferramentas de DevOps

Kubernetes:

Orquestração de contêineres para gerenciar grandes volumes de aplicativos em contêineres.

Ansible/Terraform:

Automação de provisionamento de infraestrutura e configuração.

Git:

Controle de versão distribuído.

Universidade Cruzeiro do Sul

Desafios e Benefícios de DevOps

DESAFIOS

Cultura Organizacional: A transição para DevOps pode ser desafiadora em empresas com cultura tradicional e compartimentos isolados.

Ferramentas e Automação: Implementar a automação completa exige um bom entendimento das ferramentas e processos, o que pode demandar tempo e recursos.

Desafios e Benefícios de DevOps

BENEFÍCIOS

Redução do Time-to-Market: Lançamentos mais rápidos e frequentes.

Qualidade do Software: Melhor teste e validação do código.

Resiliência: Melhor capacidade de responder a falhas e recuperação mais rápida.

DevOps

Universidade Cruzeiro do Sul

Exercício

Objetivo: Simular o fluxo DevOps com papel e caneta.

Atividade: Projeto "Infraestrutura e Configuração Manual“

Aplicação Web(Backend NodeJS, Frontend ReactJS, Banco de Dados MySQL

Grupos: 5 pessoas

Equipe Terraform: "Provisiona" servidores em papel (desenham VMs, redes).

Equipe Ansible: "Configura" servidores (lista comandos para instalar serviços).

Universidade Cruzeiro do Sul

Exercício - solução

Desenho para o Papel

[Servidor Web]
- IP: 192.168.1.10
- CPU: 2 núcleos
- RAM: 4GB
- SO: Ubuntu 22.04
- Tipo: t2.medium

[Servidor Banco de Dados]
- IP: 192.168.1.20
- CPU: 2 núcleos
- RAM: 4GB
- SO: Ubuntu 22.04
- Tipo: t2.medium

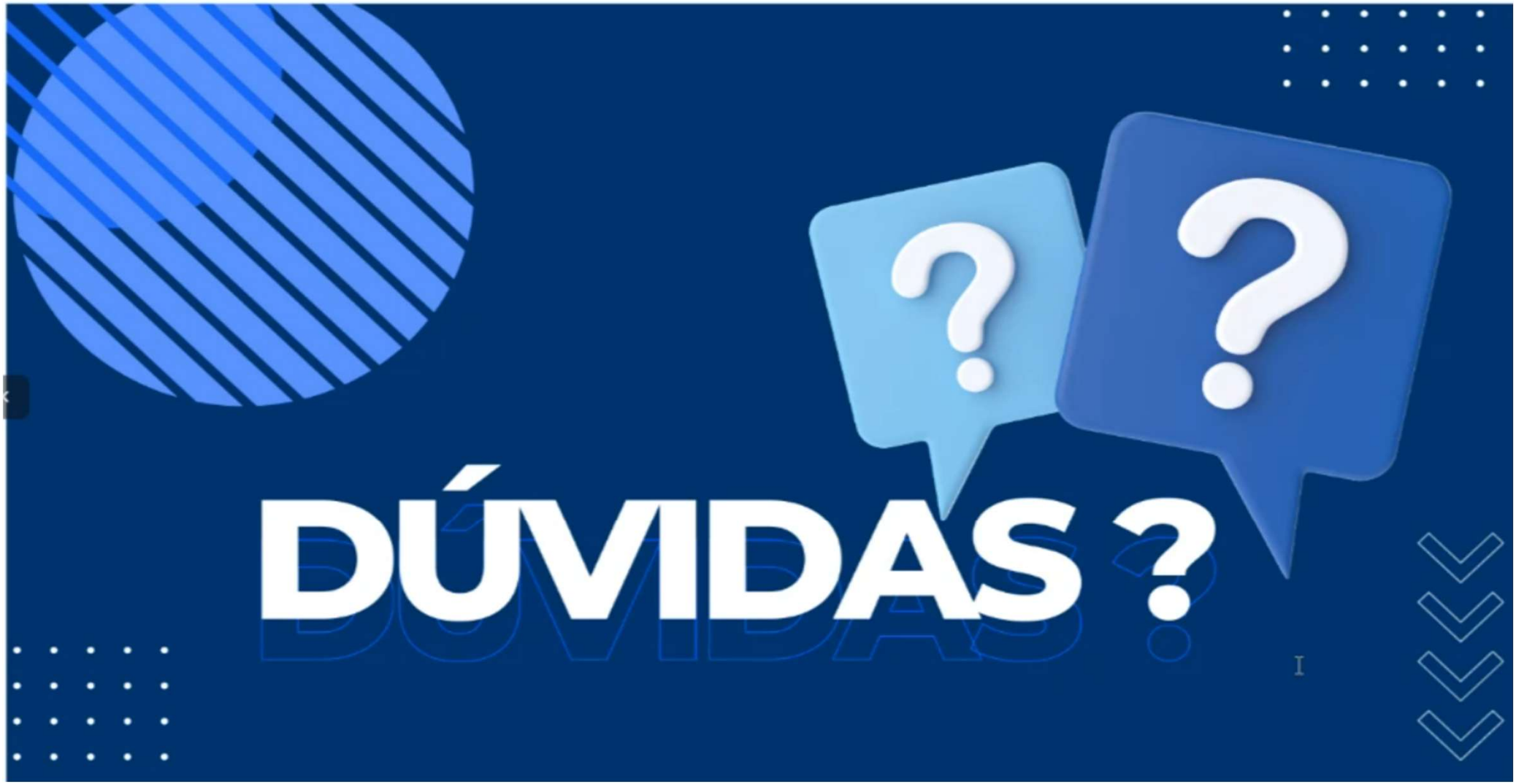
Universidade Cruzeiro do Sul

Exercício - solução

Terraform – Código simulado

```
# Arquivo: main.tf
resource "aws_instance" "web_server" {
  ami           = "ami-0c55b159cbfafe1f0"  # Ubuntu 22.04
  instance_type = "t2.medium"
  tags          = { Name = "Web Server" }
}

resource "aws_instance" "db_server" {
  ami           = "ami-0c55b159cbfafe1f0"  # Ubuntu 22.04
  instance_type = "t2.medium"
  tags          = { Name = "DB Server" }
}
```


DÚVIDAS ?